

Engenharia de Domínio: construção de artefatos reutilizáveis e análise de domínio e variabilidades

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Contexto e Motivação
- Engenharia de Domínio: Conceitos Fundamentais
- Atividades Principais
- Artefatos Reutilizáveis: Conceito e Tipos
- Exemplos Reais de Reuso
- Boas Práticas na Construção de Artefatos Reutilizáveis
- Engenharia de Domínio vs Engenharia de Aplicação
- Relação com Ecossistemas de Software
- Como a Engenharia de Domínio se Conecta

Agenda

- Conceito de Análise de Domínio
- Atividades Típicas da Análise de Domínio
- Exemplo Prático (Feature Model Simplificado)
- Variabilidades no Domínio
- Representação da Variabilidade
- Métodos de Análise de Domínio
- Aplicações Práticas

Contexto e Motivação

- **Problema no desenvolvimento de software tradicional:**
 - Projetos começam “do zero” repetidamente.
 - Muitas soluções são desenvolvidas várias vezes em contextos diferentes (duplicação de esforço).
 - Dificuldade em manter consistência e qualidade em sistemas do mesmo domínio.

Contexto e Motivação

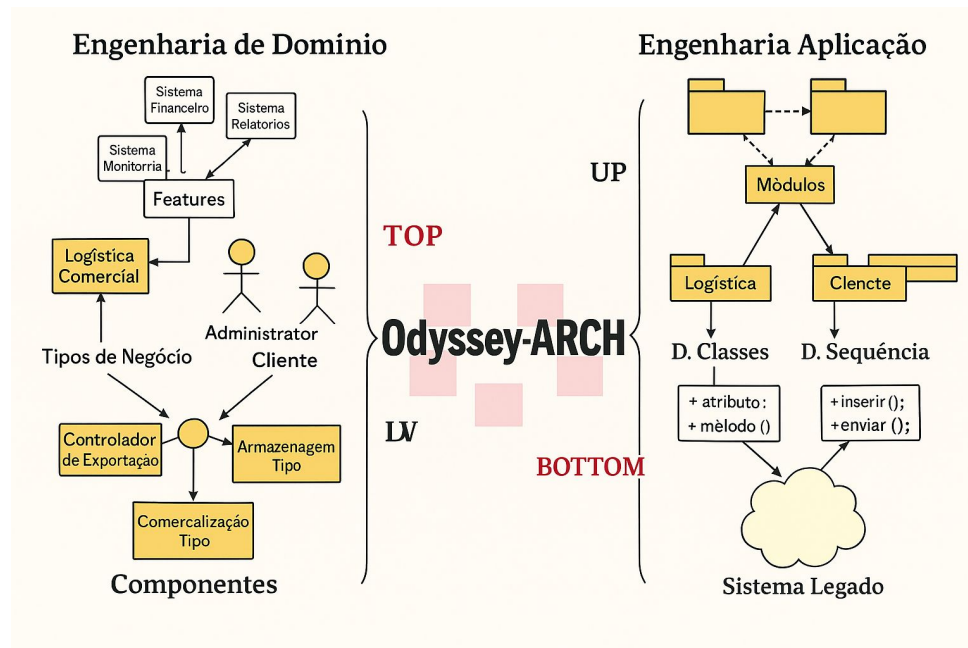
- **Solução proposta pela Engenharia de Domínio:**
 - Capturar conhecimento comum de um domínio.
 - Criar ativos reutilizáveis (artefatos de software que servem a várias aplicações).
 - Reduzir custo, tempo e risco.

Contexto e Motivação

- **Relevância atual:**
 - Com a evolução de arquiteturas orientadas a serviços, APIs públicas e frameworks, a Engenharia de Domínio se tornou um pilar para ecossistemas de software e linhas de produto de software (SPL – Software Product Lines).

Engenharia de Domínio: Conceitos Fundamentais

- A Engenharia de Domínio é uma abordagem da Engenharia de Software que visa o aumento do reuso no desenvolvimento de sistemas, focando na criação de modelos e componentes genéricos de um domínio de aplicação para reduzir custos e tempo.



Definição

- A Engenharia de Domínio (Domain Engineering – DE) é o processo sistemático de identificar, modelar e implementar os elementos comuns e variáveis de uma família de sistemas dentro de um domínio específico.
- Ela busca abstrair e estruturar o conhecimento sobre um domínio de aplicação, criando ativos reutilizáveis que aceleram e melhoram o desenvolvimento de software.

O que é um Domínio?

- Um domínio é um conjunto de problemas, conceitos e soluções que pertencem a uma área de interesse específica.
 - É o "universo de discurso" no qual um software se insere.
 - Engloba terminologia, processos de negócio, restrições, modelos e boas práticas.

- Exemplos de Domínios
 - Comércio eletrônico → carrinho de compras, catálogo, pagamentos.
 - Sistemas bancários → contas, transações, segurança.
 - Saúde → pacientes, prontuários, exames, prescrições.
 - Transporte público → rotas, pontos de parada, bilhetagem.

Objetivos da Engenharia de Domínio

- **Reutilização sistemática:** evitar reinventar a roda, maximizando a utilização de ativos de software já existentes.
- **Criação de artefatos robustos:** desenvolver modelos, componentes e frameworks que sirvam de base para várias aplicações.
- **Comunicação efetiva:** estabelecer uma linguagem comum entre engenheiros de software e especialistas do domínio.
- **Redução de custos e tempo:** acelerar o ciclo de desenvolvimento de novos sistemas dentro do mesmo domínio.
- **Aumento da qualidade:** diminuir falhas e inconsistências pela reutilização de ativos testados e validados.

Atividades Principais

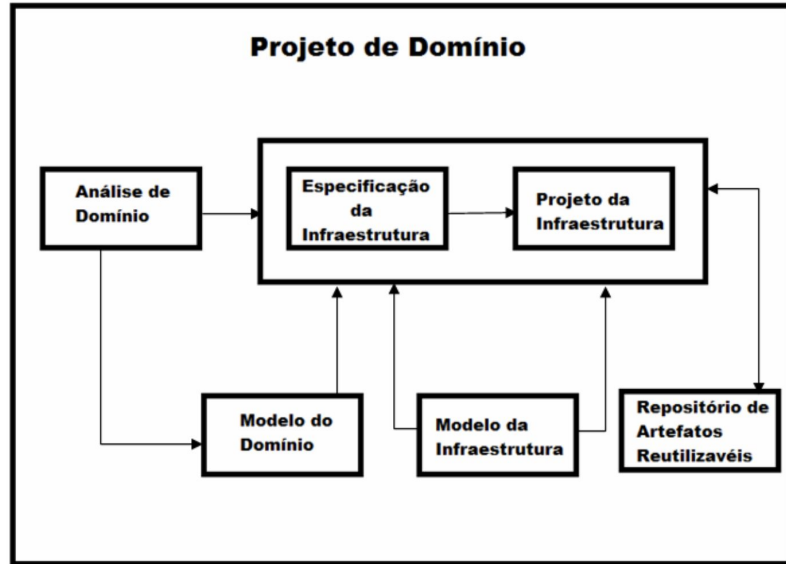


Figura 2. Modelo genérico de desenvolvimento para/com reuso.

Artigo: A Utilização de Engenharia de Domínio para Descoberta de Padrões de Análise em Sistemas de Controle de Mudanças

Autores: Ingrid Sathler Corrêa Lima, Evaldo de Oliveira da Silva

<https://seer.uniacademia.edu.br/index.php/cesi/article/view/298>

Este artigo busca através de técnicas como o reuso, engenharia de domínio, análise de domínio descrever e catalogar boas práticas e padrões conceituais, para futuras implementações de sistemas de controle de mudanças, com o propósito de facilitar, reduzir tempo e custos para a construção de sistemas de um mesmo domínio

Atividades Principais

- Análise de Domínio
- Modelagem de Domínio
- Implementação de Domínio
- Manutenção e Evolução

Análise de Domínio

- Identificação de requisitos comuns e variáveis no domínio.
- Técnicas utilizadas:
 - Análise de Features (Feature-Oriented Domain Analysis – FODA).
 - Modelagem de Variabilidade: identificar pontos em que sistemas podem divergir.
 - Revisão de sistemas existentes no domínio.
- Resultado esperado: modelos de features, catálogos de requisitos, taxonomias.

Modelagem de Domínio

- Representação formal das similaridades e variabilidades.
- Ferramentas: diagramas UML, árvores de features, metamodelos.
- Exemplo:
 - Funcionalidades obrigatórias: autenticação, CRUD básico.
 - Funcionalidades variáveis: tipos de pagamento, métodos de entrega, integrações externas.

Implementação

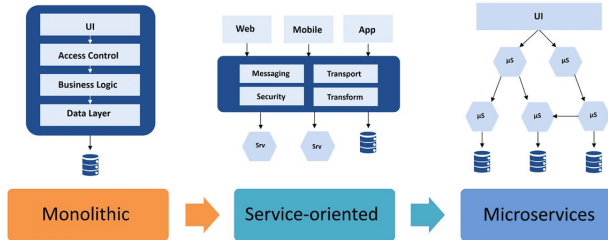
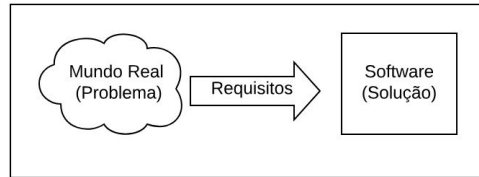
- Desenvolvimento dos ativos reutilizáveis:
 - Frameworks de software.
 - Componentes parametrizáveis.
 - Bibliotecas de classes.
 - Serviços em nuvem (APIs REST, microserviços).

Manutenção e Evolução / Validação

- Atualizar modelos e artefatos conforme o domínio evolui.
- Garantir a compatibilidade com novas demandas e tecnologias.
- Testar se os artefatos realmente atendem aos sistemas dentro do domínio.
- Critérios: cobertura de variabilidades, qualidade, facilidade de uso.

Artefatos Reutilizáveis: Conceito e Tipos

- O que é um Artefato Reutilizável?
 - Qualquer elemento produzido durante o ciclo de vida de software projetado para ser usado em mais de um projeto.



Artefatos Reutilizáveis: Conceito e Tipos

- **Requisitos reutilizáveis:** casos de uso genéricos.
- **Modelos de design:** padrões arquiteturais (MVC, camadas, microserviços).
- **Componentes de software:** módulos independentes e bem definidos.
- **Frameworks:** estruturas que provêm extensibilidade (Spring Boot, React).
- **Bibliotecas:** conjuntos de funções genéricas (NumPy, Pandas, jQuery).
- **APIs:** serviços consumíveis (Google Maps, Firebase, Twilio).
- **Linhas de Produto de Software (SPLs):** famílias de sistemas baseados em core assets.

Exemplos Reais de Reuso

- **Frameworks de desenvolvimento:**
 - Spring Boot (Java): reuso de infraestrutura.
 - Django (Python): reuso de ORM, autenticação, rotas.
 - React (JS): reuso de componentes de UI.
- **APIs:**
 - Google Maps (localização e rotas).
 - Firebase (autenticação, banco de dados, push).
 - Stripe (pagamentos).
- **Arquiteturas de referência:**
 - MVC (Model-View-Controller).
 - Microserviços.
 - Clean Architecture.

Características de Artefatos Reutilizáveis

- **Generalidade:** devem atender múltiplos contextos do domínio.
- **Flexibilidade:** devem permitir parametrização/extensão.
- **Documentação clara:** essencial para adoção.
- **Independência:** baixo acoplamento em relação a outros módulos.
- **Qualidade e confiabilidade:** testados, estáveis e validados em diferentes cenários.

Boas Práticas na Construção de Artefatos Reutilizáveis

- Identificação clara do domínio e de seus limites.
- Design modular (alta coesão, baixo acoplamento).
- Documentação e exemplos práticos de uso.
- Testes abrangentes e manutenção contínua.
- Suporte à extensibilidade (hooks, configurações, interfaces bem definidas).
- Uso de padrões de projeto (Factory, Strategy, Adapter, etc.).
- Adoção de versionamento semântico (semver) para compatibilidade.

Benefícios

- Econômicos: redução de custo de desenvolvimento a médio/longo prazo.
- Produtividade: novos sistemas são construídos mais rápido usando ativos existentes.
- Qualidade: ativos reutilizados já foram testados e validados.
- Consistência: sistemas dentro de um domínio seguem padrões semelhantes.
- Inovação: engenheiros podem se concentrar em funcionalidades novas, não em reinventar o básico.

Desafios

- Alto custo inicial para construir ativos genéricos.
- Complexidade de modelagem de variabilidades.
- Resistência organizacional ao reuso sistemático.
- Manutenção de ativos reutilizáveis (documentação, evolução, compatibilidade).
- Granularidade do reuso:
 - Muito genérico → difícil de aplicar.
 - Muito específico → pouco reaproveitável.

Engenharia de Domínio vs Engenharia de Aplicação

Aspecto	Engenharia de Domínio	Engenharia de Aplicação
Foco	Criar ativos reutilizáveis (frameworks, componentes, modelos)	Usar ativos para construir sistemas específicos
Produto	Core assets, frameworks, SPLs	Aplicações finais
Exemplo	Criar um framework de e-commerce	Desenvolver uma loja online usando o framework

Futuro da Engenharia de Domínio

- **Integração com DevOps:** ativos reutilizáveis entregues como pacotes, contêineres, microserviços.
- **Evolução para ecossistemas:** foco não apenas em empresas, mas em comunidades open source (ex.: React, TensorFlow).
- **Apoio de Inteligência Artificial:** geração de componentes genéricos a partir de análise de grandes bases de código.
- **Tendência a plataformas low-code/no-code:** onde ativos reutilizáveis são abstraídos para usuários finais.

Relação com Ecossistemas de Software

- Um ecossistema de software é um ambiente no qual múltiplas organizações, comunidades e indivíduos interagem em torno de uma plataforma ou tecnologia central.
- Ele é composto por:
 - Plataforma base (núcleo tecnológico, ex.: Android, iOS, AWS).
 - Provedores de extensões (apps, plugins, serviços adicionais).
 - Usuários finais que consomem os produtos.
 - Comunidade de desenvolvedores e parceiros que contribuem para evolução.
- Analogia biológica: assim como em um ecossistema natural, os atores convivem, competem e colaboram para manter o ambiente vivo e em crescimento.

Relação com Ecossistemas de Software

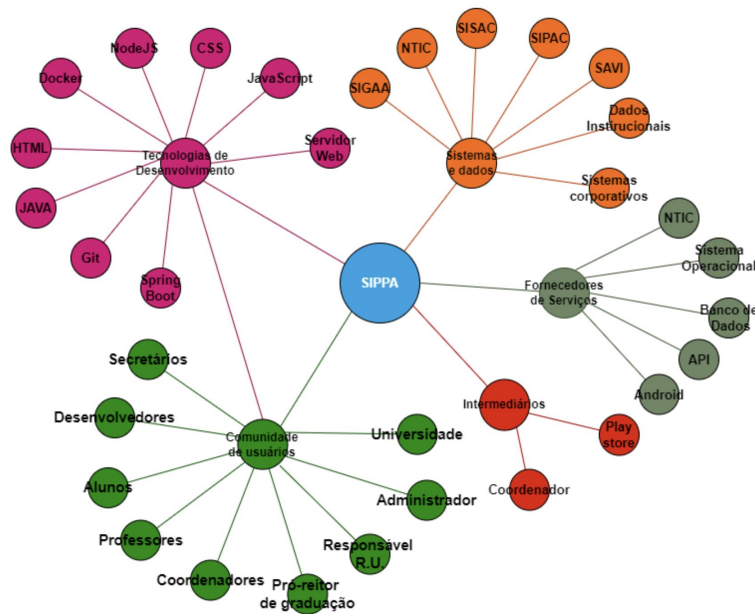


Figura 3. Rede sociotécnica para o ECOS SIPPA.

Artigo: Estudo Preliminar sobre a Modelagem de Ecossistemas de Software: O Caso do ECOS SIPPA

Autores: Francisco Victor da S. Pinheiro, Maria Erilane L. da Silva, Ronier da Silva Lima, Emanuel Ferreira Coutinho, Carla I. M. Bezerra e Rossana M. C. Andrade

<https://sol.sbc.org.br/index.php/sbsc/article/view/35802>

Este artigo tem como objetivo apresentar a modelagem SSN do ECOS SIPPA, um aplicativo de gestão acadêmica. Foram identificados, de forma preliminar, alguns elementos de modelagem presentes na literatura sobre ECOS, os quais foram explorados no contexto de um ecossistema real, no domínio educacional.

Relação com Ecossistemas de Software

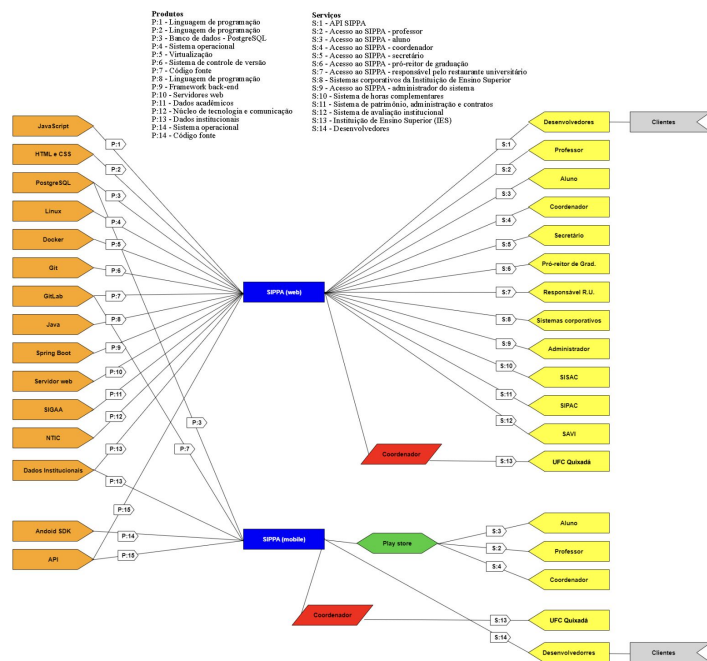


Figura 4. Modelagem SSN para o ECOS SIPPA.

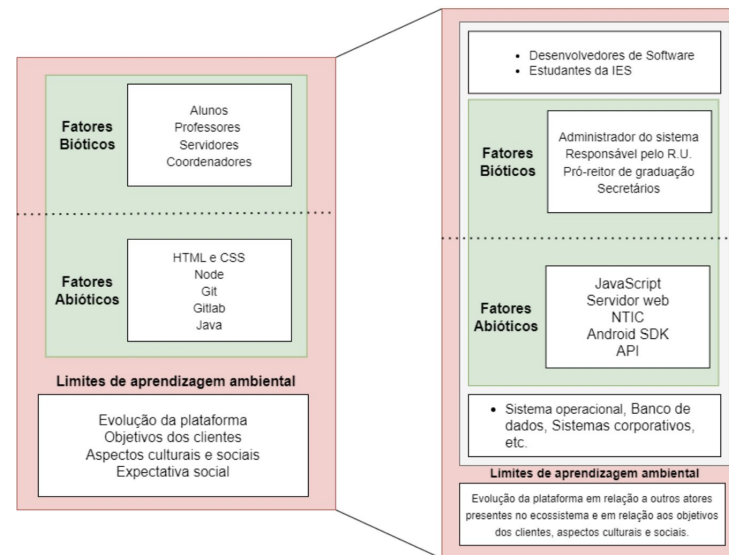
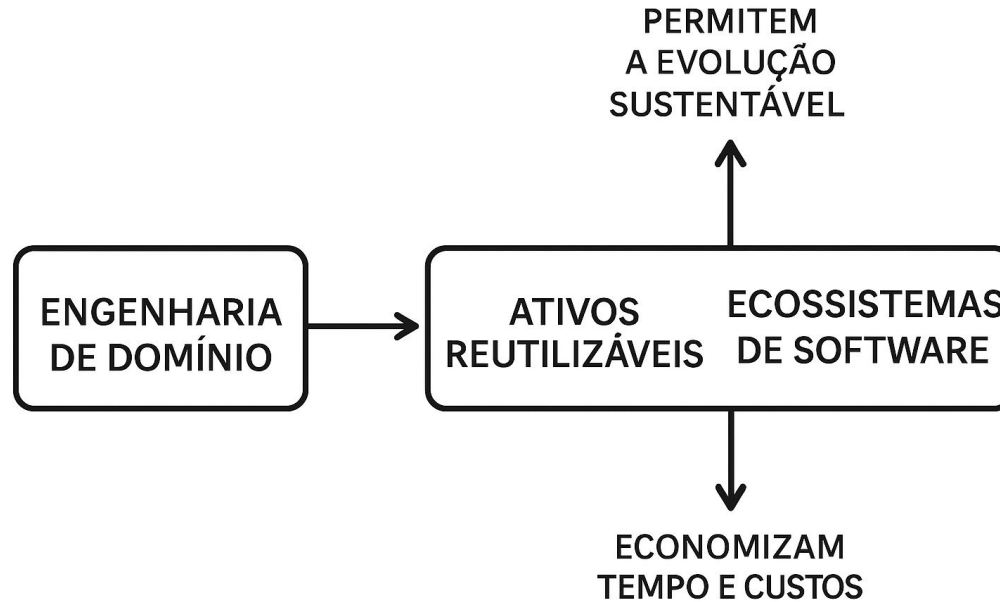


Figura 5. Visão geral do ECOS SIPPA.

Como a Engenharia de Domínio se Conecta

- A Engenharia de Domínio atua como um mecanismo de sustentação dentro de ecossistemas de software, pois:
 - Cria modelos conceituais que estabelecem linguagem comum entre diferentes atores do ecossistema.
 - Define ativos reutilizáveis (frameworks, bibliotecas, APIs, DSLs) que permitem consistência nas soluções.
 - Suporta variabilidade, garantindo que diferentes aplicações possam se adaptar a contextos distintos sem perder qualidade.
 - Permite evolução sustentável, reduzindo custos e mantendo compatibilidade entre versões.

Como a Engenharia de Domínio se Conecta



Exemplos Práticos

- **Android:**
 - O Android SDK é um ativo de domínio essencial.
 - Fornece APIs, bibliotecas e componentes reutilizáveis.
 - Permite que milhares de desenvolvedores criem aplicativos consistentes e compatíveis.

- **E-commerce (ex.: Magento, Shopify, VTEX):**
 - A plataforma fornece ativos como carrinho de compras, meios de pagamento e catálogo.
 - Lojas virtuais reutilizam esses componentes, variando apenas em design, integrações e regras de negócio.

- **Open Source (ex.: Ecossistema do Python):**
 - O núcleo da linguagem é um ativo de domínio.
 - Frameworks como Django e Flask são reutilizados por milhares de sistemas.
 - A evolução é colaborativa e sustentável.

Benefícios da Conexão entre Engenharia de Domínio e Ecossistemas

- **Padronização:** todos os participantes seguem modelos comuns.
- **Reuso em larga escala:** acelera o desenvolvimento de novas soluções.
- **Inovação sobre bases sólidas:** empresas podem focar em diferenciais sem reinventar o básico.
- **Sustentabilidade:** facilita manutenção e evolução do ecossistema.
- **Escalabilidade:** mais participantes podem entrar no ecossistema sem comprometer qualidade.

“Se a Engenharia de Domínio organiza o conhecimento, os Ecossistemas de Software ampliam esse conhecimento em escala global, tornando o reuso não apenas uma prática interna, mas uma estratégia de sobrevivência e evolução coletiva.”

Resumo

- A Engenharia de Domínio é um pilar essencial da Engenharia de Software moderna, permitindo:
 - maior reutilização sistemática,
 - melhor comunicação com especialistas,
 - e redução de custos e riscos no desenvolvimento.
- Ela conecta práticas de modelagem, reutilização e variabilidade para sustentar tanto linhas de produto de software quanto ecossistemas de software.

Motivação

- **Problema:** Organizações desenvolvem sistemas semelhantes em um mesmo domínio (ex.: apps bancários, ERPs, plataformas de ensino).
- **Questão:** como evitar retrabalho e aproveitar o conhecimento já existente?
- **Exemplo prático:**
 - Bancos digitais → todos têm cadastro, consulta de saldo, transferências.
 - O que varia? Pix, cartão virtual, criptomoedas.

Motivação

“Quais sistemas do dia a dia vocês percebem que têm versões ou opções diferentes?”

Conceito de Análise de Domínio

- **Definição:** processo de estudar, organizar e formalizar o conhecimento sobre um domínio de aplicação, identificando tanto os requisitos comuns quanto as variabilidades.
- **Objetivo central:** criar uma visão de referência (modelos, padrões, arquiteturas) que guie o desenvolvimento de múltiplos sistemas no mesmo domínio.

Motivação da Análise de Domínio

- Evitar retrabalho no desenvolvimento de sistemas semelhantes.
- Capturar de forma explícita o que é comum e o que é variável em uma família de aplicações.
- Permitir reuso sistemático, indo além do reuso ad hoc (copiar e colar código).
- Basear a engenharia de software em modelos de referência confiáveis.

Propósito da Análise de Domínio

- Fornecer um entendimento compartilhado entre analistas, arquitetos e desenvolvedores.
- Apoiar a criação de linhas de produto de software (Software Product Lines).
- Servir como base para a definição de arquiteturas de referência.
- Auxiliar na tomada de decisão sobre variabilidades em cada projeto.

Atividades Típicas da Análise de Domínio

- Levantamento de requisitos comuns
- Mapeamento de variabilidades
- Definição de relacionamentos e restrições
- Criação de modelos de referência

Levantamento de requisitos comuns

- **Descrição:** Identificação de funcionalidades, regras de negócio e entidades que se repetem em todos os sistemas de um mesmo domínio.
- **Objetivo:** Construir uma base sólida que represente o “núcleo essencial” do domínio.
- **Como fazer:**
 - Entrevistas com especialistas de domínio.
 - Análise de sistemas legados.
 - Documentação de processos organizacionais.
- **Exemplo:** Em sistemas bancários, todos os sistemas precisam gerenciar contas, clientes e transações, independentemente do porte do banco ou canal (web, mobile, caixa eletrônico).
- **Benefício:** Evita que cada projeto reinvente o básico e garante consistência.

Mapeamento de Variabilidades

- **Descrição:** Identificação de funcionalidades que mudam de acordo com o contexto, cliente ou mercado.
- **Objetivo:** Reconhecer os pontos onde os sistemas podem se diferenciar sem comprometer a estrutura comum.
- **Tipos de variabilidade:**
 - Opcional: recurso pode estar presente ou não (ex.: relatórios avançados).
 - Alternativa: escolher entre opções mutuamente exclusivas (ex.: tipo de autenticação: senha, biometria, token).
 - Parametrizada: ajuste por configuração (ex.: limite máximo de usuários por plano).
- **Exemplo:** Em e-commerce, o método de pagamento pode variar: cartão de crédito, boleto, Pix, PayPal.
- **Benefício:** Flexibilidade para atender diferentes clientes sem refazer a base do sistema.

Definição de Relacionamentos e Restrições

- **Descrição:** Estabelecer como os elementos comuns e variáveis podem ser combinados.
- **Objetivo:** Evitar combinações inválidas e orientar decisões de configuração dos sistemas.
- **Tipos de restrições:**
 - **Obrigatórias:** uma escolha implica obrigatoriamente outra (ex.: se houver biometria, deve haver módulo de cadastro de impressões digitais).
 - **Excludentes:** duas opções não podem coexistir (ex.: login anônimo e login obrigatório).
 - **Dependências:** presença de uma funcionalidade depende de outra (ex.: pagamento via Pix depende de integração com banco).
- **Exemplo:** Em um sistema de autenticação, se biometria estiver habilitada, pode ser permitido dispensar a senha simples.
- **Benefício:** Garante consistência entre as soluções e reduz erros de configuração.

Criação de Modelos de Referência

- **Descrição:** Representação formal do conhecimento levantado, servindo como base para desenvolvimento e documentação.
- **Formas de representação:**
 - Feature Models (Modelos de Características): diagramas que mostram o que é comum e o que é variável.
 - Arquiteturas de Referência: padrões estruturais que orientam o design técnico.
 - Catálogos de componentes reutilizáveis: listagem organizada de artefatos (código, módulos, templates).
- **Exemplo:** Um feature model de e-commerce pode mostrar que “Carrinho de Compras” é obrigatório, enquanto “Lista de Desejos” é opcional.
- **Benefício:** Serve como guia mestre para futuros desenvolvimentos, garantindo padronização e reuso sistemático.

Exemplo Prático (Feature Model Simplificado)

- **Domínio:** Aplicações de E-commerce
 - **Comum:** catálogo de produtos, carrinho de compras, checkout.
 - **Variável:** forma de pagamento (cartão, Pix, PayPal).
 - **Restrição:** se Pix estiver habilitado, precisa ter integração bancária.

Benefícios da Análise de Domínio

- **Redução de custos:** menor esforço de manutenção e evolução.
- **Maior qualidade:** consistência entre sistemas da mesma família.
- **Agilidade:** desenvolvimento mais rápido de novos produtos.
- **Flexibilidade:** adaptação a diferentes clientes e mercados.
- **Reuso sistemático:** aproveitamento de modelos, código e testes.

Desafios da Análise de Domínio

- Exige investimento inicial significativo.
- Depende de especialistas no domínio.
- Risco de excesso de variabilidade, tornando modelos complexos.
- Precisa de ferramentas adequadas para documentar e manter modelos.

Resumo

- A análise de domínio é um passo fundamental para a engenharia de software orientada ao reuso.
- Permite identificar o que é comum e o que pode variar dentro de um domínio.
- Resulta em modelos de referência que servem de base para múltiplos projetos.
- Apesar dos desafios, seus benefícios superam os custos, especialmente em organizações que desenvolvem muitas soluções semelhantes.

Variabilidades no Domínio

- **Definição:** capacidade de um sistema ou artefato ser adaptado/customizado.
- **Por que é importante?**
 - Clientes e contextos diferentes exigem funcionalidades distintas.
 - Reuso só funciona se soubermos onde é fixo e onde pode variar.

Tipos de variabilidade

- **Funcional** – diferentes opções de funcionalidades.
- **Arquitetural** – arranjos de componentes diferentes.
- **Tecnológica** – alternativas de frameworks, bancos, linguagens.
- **Regras de negócio** – mudanças em políticas ou fluxos.

Variabilidade Funcional

- **Descrição:** Refere-se às diferenças nas funcionalidades que um sistema pode oferecer.
- **Quando ocorre:** Quando diferentes clientes ou cenários de uso exigem opções de recursos distintos.
- **Exemplos:**
 - Login via senha (básico).
 - Login por biometria (mais seguro).
 - Login por redes sociais (mais prático).
- **Impacto:** Afeta diretamente a experiência do usuário e a cobertura de requisitos.
- **Benefício da análise:** Permite projetar sistemas modulares, onde novas funcionalidades podem ser ativadas ou desativadas sem refazer todo o software.

Variabilidade Arquitetural

- **Descrição:** Diferenças no arranjo estrutural dos componentes de software.
- **Quando ocorre:** Quando a mesma solução pode ser implementada em diferentes arquiteturas, dependendo de requisitos não funcionais (desempenho, escalabilidade, manutenibilidade).
- **Exemplos:**
 - Arquitetura monolítica: todos os módulos integrados em uma única aplicação.
 - Arquitetura baseada em microsserviços: cada funcionalidade encapsulada em serviços independentes.
- **Impacto:** Afeta escalabilidade, manutenção e custo operacional.
- **Benefício da análise:** Permite escolher a arquitetura mais adequada sem perder compatibilidade com os elementos comuns do domínio.

Variabilidade Tecnológica

- **Descrição:** Refere-se às diferentes tecnologias, ferramentas ou frameworks que podem ser usadas para implementar um sistema.
- **Quando ocorre:** Quando o mesmo requisito pode ser atendido por diferentes soluções tecnológicas, cada uma com prós e contras.
- **Exemplos:**
 - Banco de dados: PostgreSQL (relacional) vs MongoDB (NoSQL).
 - Frameworks web: Django (Python) vs Spring Boot (Java).
 - Front-end: React vs Angular vs Vue.
- **Impacto:** Afeta tempo de desenvolvimento, curva de aprendizado da equipe e custo de manutenção.
- **Benefício da análise:** Evita dependência de uma única tecnologia e facilita a portabilidade entre ambientes.

Variabilidade em Regras de Negócio

- **Descrição:** Diferenças nas políticas, legislações ou fluxos organizacionais que impactam diretamente as funcionalidades.
- **Quando ocorre:** Quando o mesmo domínio precisa atender contextos legais ou culturais diferentes.
- **Exemplos:**
 - Cálculo de impostos varia entre países (ICMS no Brasil, IVA na Europa, Sales Tax nos EUA).
 - Fluxo de aprovação em empresas: algumas exigem uma assinatura, outras exigem múltiplas validações.
- **Impacto:** Afeta diretamente a lógica de negócio e pode exigir adaptações específicas em cada cliente.
- **Benefício da análise:** Permite criar sistemas configuráveis e compatíveis com diferentes legislações ou contextos organizacionais.

Representação da Variabilidade

- Feature Models (Modelos de Características) – representação gráfica em árvore.
 - Feature obrigatória → sempre presente.
 - Feature opcional → pode ou não estar presente.
 - Alternativas → escolha entre opções.
 - Grupos OR → pode escolher várias opções.

Exemplo – Sistema de E-commerce:

E-commerce

- |— Catálogo de Produtos (obrigatório)
- |— Carrinho de Compras (obrigatório)
- |— Pagamento
 - | |— Pix (opcional)
 - | |— Cartão de Crédito (opcional)
 - | |— PayPal (opcional)
- |— Entrega
 - |— Retirada na Loja (alternativa)
 - |— Correios (alternativa)
 - |— Transportadora (alternativa)

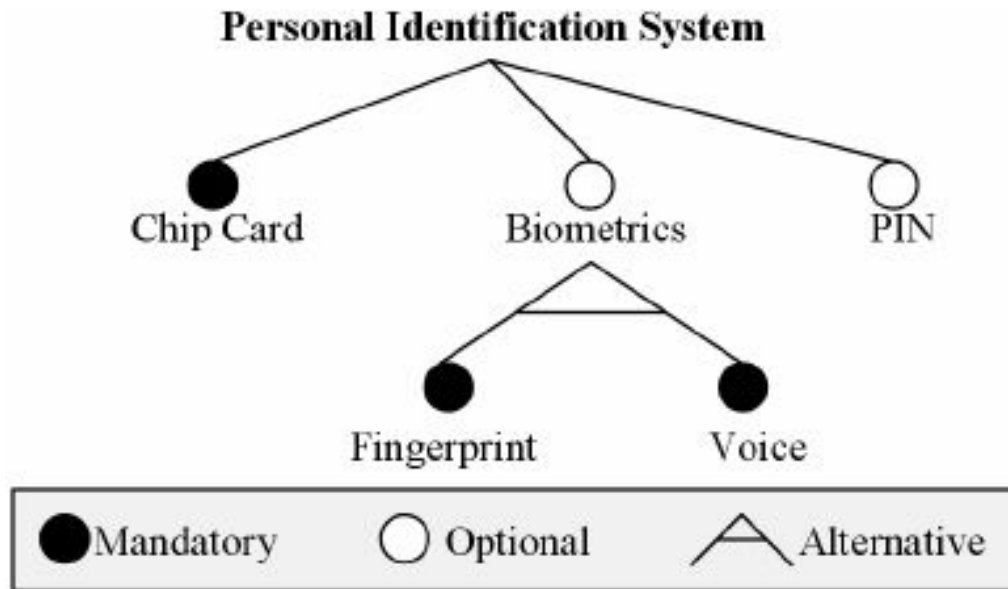
Métodos de Análise de Domínio

- **FODA (Feature-Oriented Domain Analysis)**
 - Pioneiro, introduziu feature models.
 - Foco: identificar e modelar funcionalidades comuns/variáveis.
- **FORM (Feature-Oriented Reuse Method)**
 - Extensão do FODA.
 - Leva em conta arquitetura de software reutilizável.
- **CDA (Contextual Domain Analysis)**
 - Considera o contexto do sistema (usuários, ambiente, restrições externas).

Feature-Oriented Domain Analysis

- **Origem:** Criado no final dos anos 1980 pelo SEI (Software Engineering Institute). Foi o método pioneiro na área.
- **Contribuição central:** Introduziu o conceito de feature model (modelo de características) como forma de representar o que é comum e o que é variável em um domínio.
- **Foco:**
 - Identificar funcionalidades obrigatórias, opcionais e alternativas.
 - Estruturar essas funcionalidades em árvores de características.
- **Etapas principais:**
 - Levantamento do domínio (análise de requisitos, documentos e sistemas existentes).
 - Identificação de features comuns e features variáveis.
 - Construção do Feature Model com relacionamentos (obrigatório, opcional, alternativo, restrições).
- **Exemplo:** Em e-commerce, “Carrinho de Compras” é obrigatório, enquanto “Lista de Desejos” é opcional.

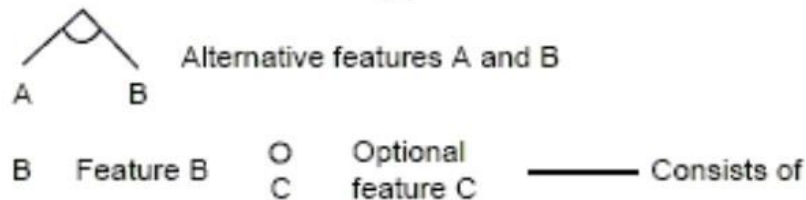
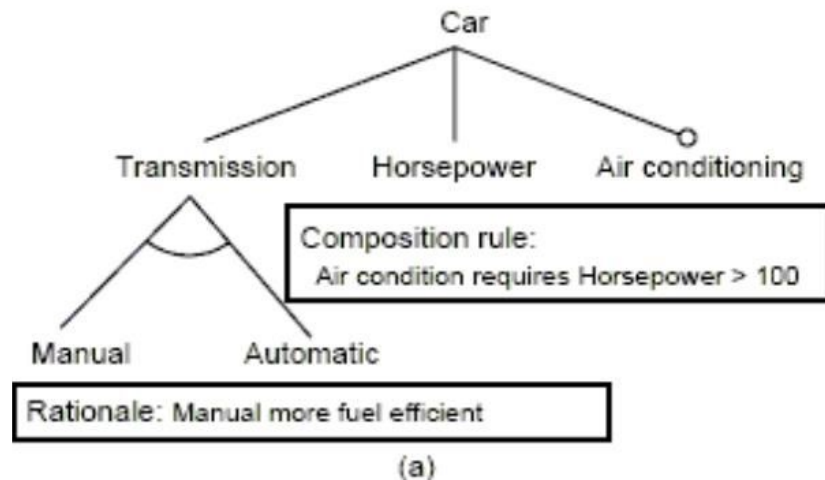
Feature-Oriented Domain Analysis



Feature-Oriented Reuse Method

- **Origem:** Desenvolvido nos anos 1990 por Kang et al., como evolução direta do FODA.
- **Contribuição central:** Leva a análise além das funcionalidades, conectando-as com a arquitetura de software reutilizável.
- **Foco:**
 - Criar não apenas modelos de features, mas também componentes de arquitetura e frameworks reutilizáveis.
 - Permitir que a variabilidade identificada em nível funcional seja refletida na arquitetura.
- **Etapas principais:**
 - Construção do Feature Model (como no FODA).
 - Mapeamento de features em componentes arquiteturais.
 - Criação de arquiteturas de referência e frameworks reutilizáveis.
- **Exemplo:** Se no domínio de e-commerce o pagamento pode variar (cartão, Pix, PayPal), no FORM isso se traduz em diferentes componentes arquiteturais de processamento de pagamento.

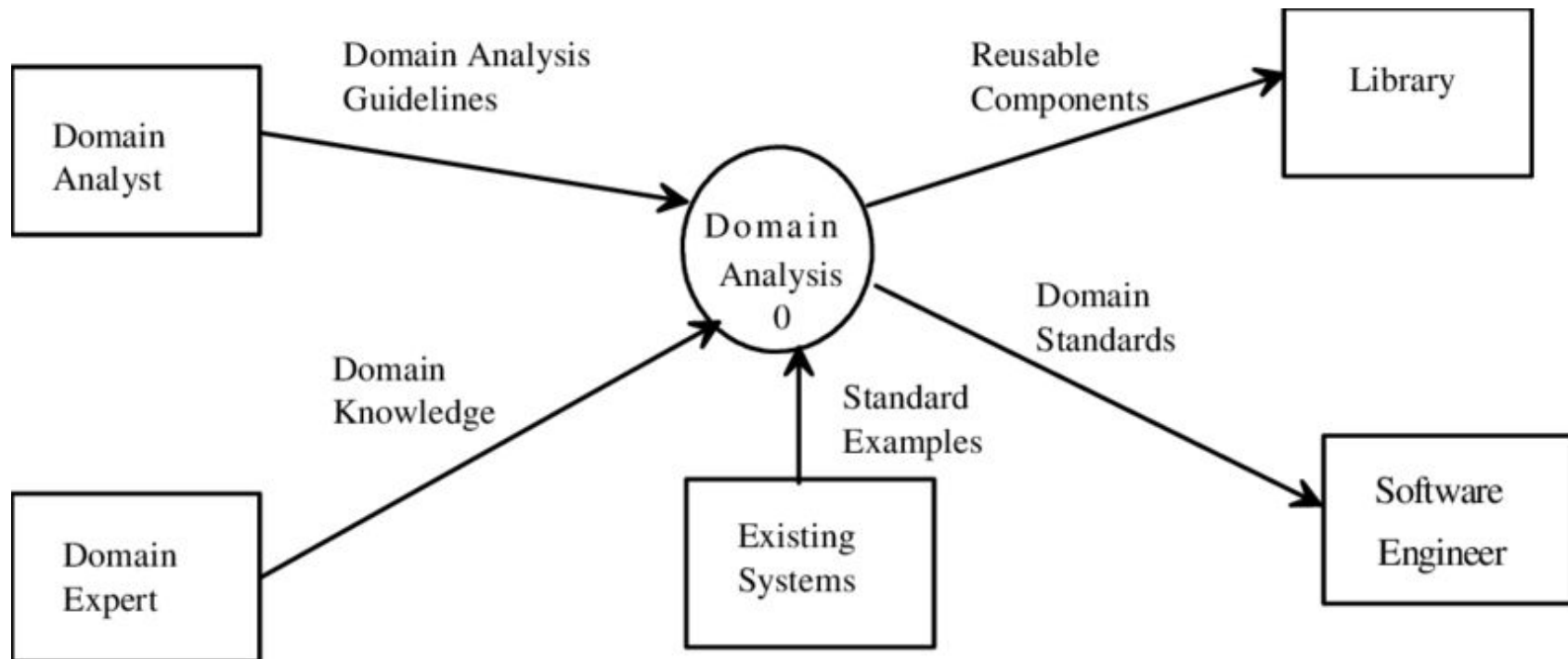
Feature-Oriented Reuse Method



Contextual Domain Analysis

- **Origem:** Surge como abordagem complementar às anteriores, trazendo uma visão mais ampla.
- **Contribuição central:** Considera o contexto de uso do sistema, indo além das funcionalidades e da arquitetura.
- **Foco:**
 - Entender quem são os usuários do sistema.
 - Identificar o ambiente de execução (hardware, rede, dispositivos).
 - Considerar restrições externas (legislação, políticas organizacionais, regulamentos).
- **Etapas principais:**
 - Analisar o contexto de aplicação (cenários reais de uso).
 - Identificar variabilidades contextuais (ex.: leis diferentes em cada país).
 - Relacionar contexto com features e componentes arquiteturais.
- **Exemplo:**
 - Um sistema de saúde precisa considerar regras legais diferentes em cada país para tratamento de dados de pacientes.
 - Em ambientes militares, um mesmo software pode ter restrições diferentes de segurança dependendo do nível de confidencialidade.

Contextual Domain Analysis



Comparação

Método	Foco Principal	Contribuição	Limitação
FODA	Funcionalidades (features)	Pioneiro, introduziu feature models	Não aborda arquitetura
FORM	Funcionalidades + Arquitetura	Extensão do FODA, cria frameworks reutilizáveis	Mais esforço inicial
CDA	Contexto (usuários, ambiente, restrições)	Traz visão contextual para variabilidades	Modelos mais complexos

Aplicações Práticas

- **Linha de Produtos Automotivos:** mesmo modelo de carro, versões básica, intermediária e premium.
- **Sistemas Bancários:** variabilidade em autenticação, métodos de pagamento, canais de acesso.
- **Aplicativos de Streaming:** comum = catálogo, busca, player; variável = planos, anúncios, download offline.

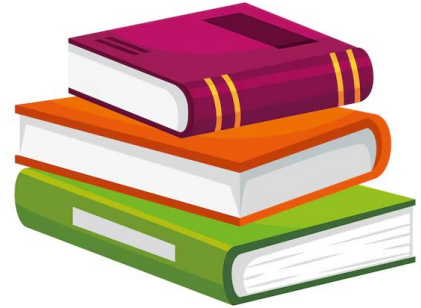
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

